Session Management

Ehsan Khamespanah

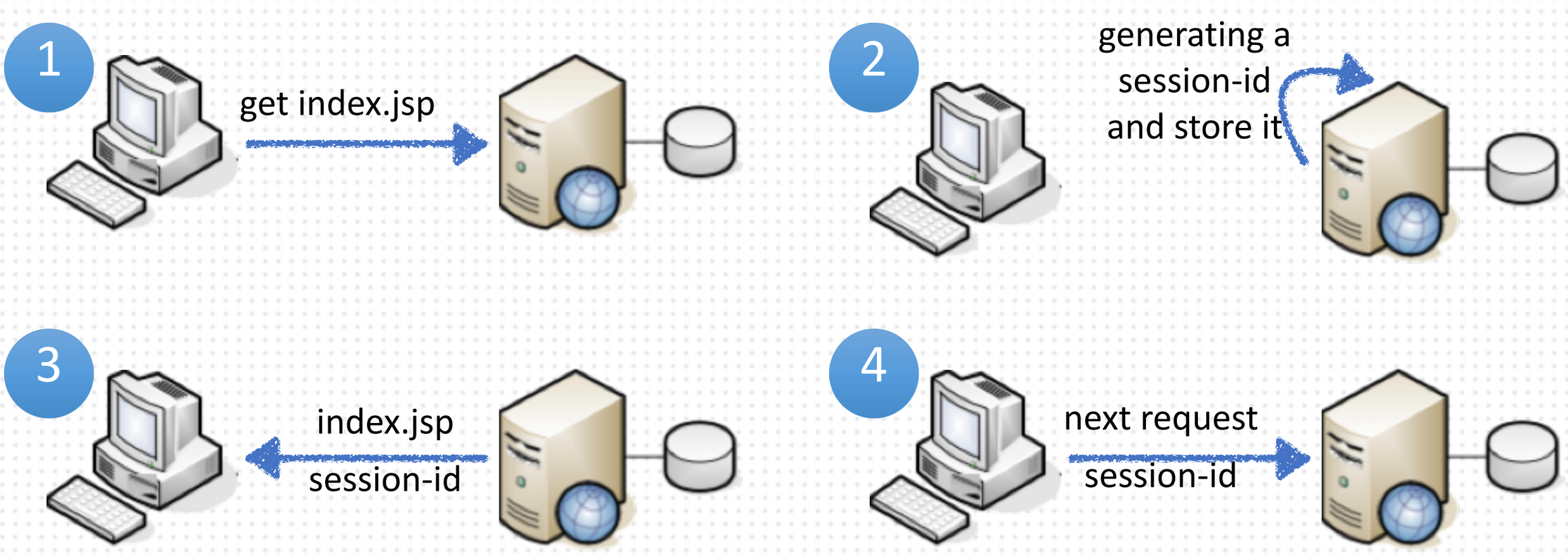
Spring 2023

What is Session?

- HTTP protocol and Web Servers are stateless
  - Every request is a new request to process
  - They can't identify if it's coming from client that has been sending request previously
    - a shopping cart application should know who is sending the request to add an item and in which cart the item has to be added
- **Session** is a conversional state between client and server and it can consists of multiple request and response between client and server

How Session Works?

- Since HTTP and Web Server both are stateless, the only way to maintain a session is when some unique information about the session is passed between server and client in every request and response.



Handling Session ID - Client Side

- URL Rewriting
  - Links which are sent from the server are rewritten to embed the session id

```
<a href="link-adr?param1=value1&param2=value2&sessionid=102987>
```

- It is not easy to overwrite all the links
- How to share a link?

Handling Session ID - Client Side

- URL Rewriting
- Hidden Fields

```
<form action="something">
    username: <input type="text" name="username">
    password: <input type="text" name="password">
    <input type="hidden" name="sessionid">
    <input type="submit">
</form>
```

Handling Session ID - Client Side

- URL Rewriting
- Hidden Fields
- Cookies: An HTTP cookie is a small piece of data sent from a website and stored on the user's computer by the user's web browser while the user is browsing
  - Server sets a set of cookie values in response to a request
  - Client puts the cookie values in all of the related requests

Cookies for google.com

```
GET / HTTPS/1.1
user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_2)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163
Safari/537.36
host: google.com
accept: text/html,application/xhtml+xml,application/...
accept-encoding: gzip, deflate, br
accept-language: en-US,en;q=0.9,fa;q=0.8
cache-control: no-cache
```

```
Request URL: https://www.google.com/
Request Method: GET
Status Code: 200
content-length: 61781
content-type: text/html; charset=UTF-8
date: Sun, 19 Apr 2020 02:21:51 GMT
set-cookie: 1P_JAR=2020-04-19-02; expires=Tue, 19-May-2020
02:21:51 GMT; path=/; domain=.google.com; Secure; SameSite=none
set-cookie: NID=202=LMRoqJHbaZ4bpfI4T5hN2YLv8JB3sCM1DYuDC5YHd;
expires=Mon, 19-Oct-2020 02:21:51 GMT; path=/; domain=.google.com;
Secure; HttpOnly; SameSite=none
```

Get Values of Cookies

- In Servelets

```
public void doGet(HttpServletRequest req, HttpServletResponse res) {
   Cookie[] cookies = req.getCookies();
   for (int i = 0; i < cookies.length; i++) {
      Cookie c = cookies[i];
      String name = c.getName();
      String value = c.getValue();
   }
}
```

- In Spring

```
@RequestMapping("/hello.html")
public String hello(
   @CookieValue(value = "foo", defaultValue = "hello") String
fooCookie) {

   //..
}
```

Set Values in Cookies

- In Servelets

```
public void doGet(HttpServletRequest req, HttpServletResponse res) {

   Cookie c = new Cookie(name, value);
   response.addCookie(c);

}
```

- In Spring

```
@RequestMapping(value = "/hello.htm")
public String hello(
   @CookieValue(value = "hitCounter", defaultValue = "0") Long
   hitCounter,HttpServletResponse response) {

      // create cookie and set it in response
      Cookie cookie = new Cookie("hitCounter", hitCounter.toString());
      response.addCookie(cookie);

      return "hello";
}
```

Handling Session ID - Client Side

- URL Rewriting
- Hidden Fields
- Cookies
- HTML5 Data Storages
  - localStorage: stores data with no expiration date
  - sessionStorage: stores data for one session

```
// Store
localStorage.setItem(“lastname", "Smith");

// Retrieve
document.getElementById("result").innerHTML =
    localStorage.getItem("lastname");
```

Handling Session ID - Server Side

- Session IDs can be stored in the server-side session storage (server local memory)

```
@RequestMapping(value = {"/login/{username}/{password}",
        "/login/{username}"}, method = RequestMethod.GET)
public Boolean doLogin(
      @PathVariable(value = "username") String username,
      @PathVariable(value = "password") String password) {

   ServletRequestAttributes attr =
      (ServletRequestAttributes) RequestContextHolder.currentRequestAttributes();
   HttpSession httpSession = attr.getRequest().getSession(true);

   // Do login logic here!

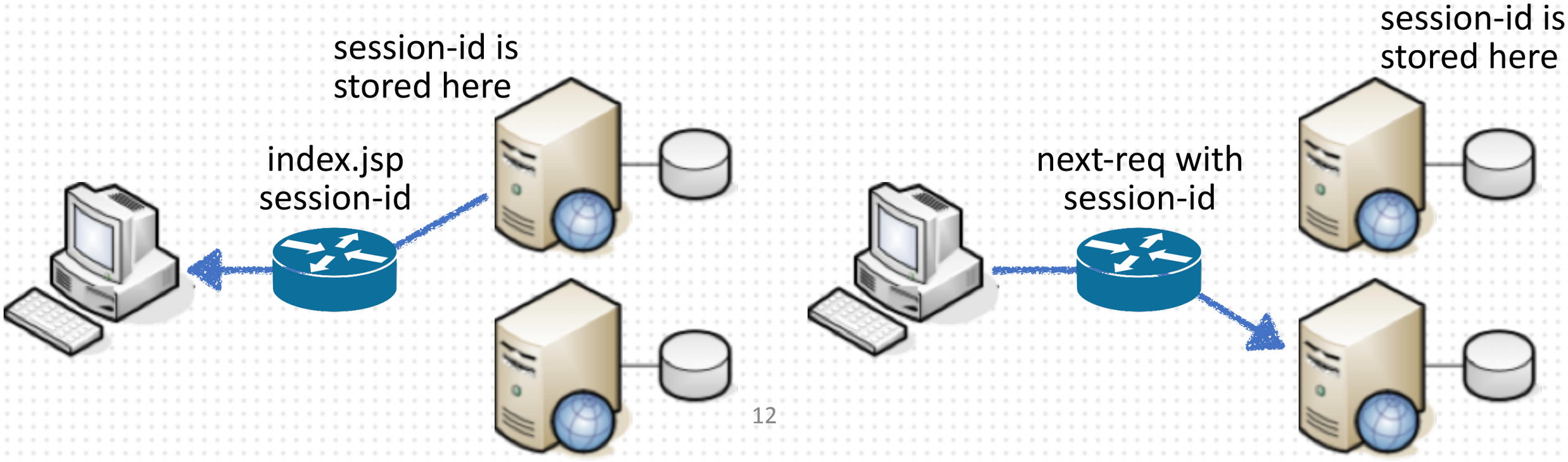
   httpSession.setAttribute("userId", userId);

   return false;
}
```

Handling Session ID - Server Side

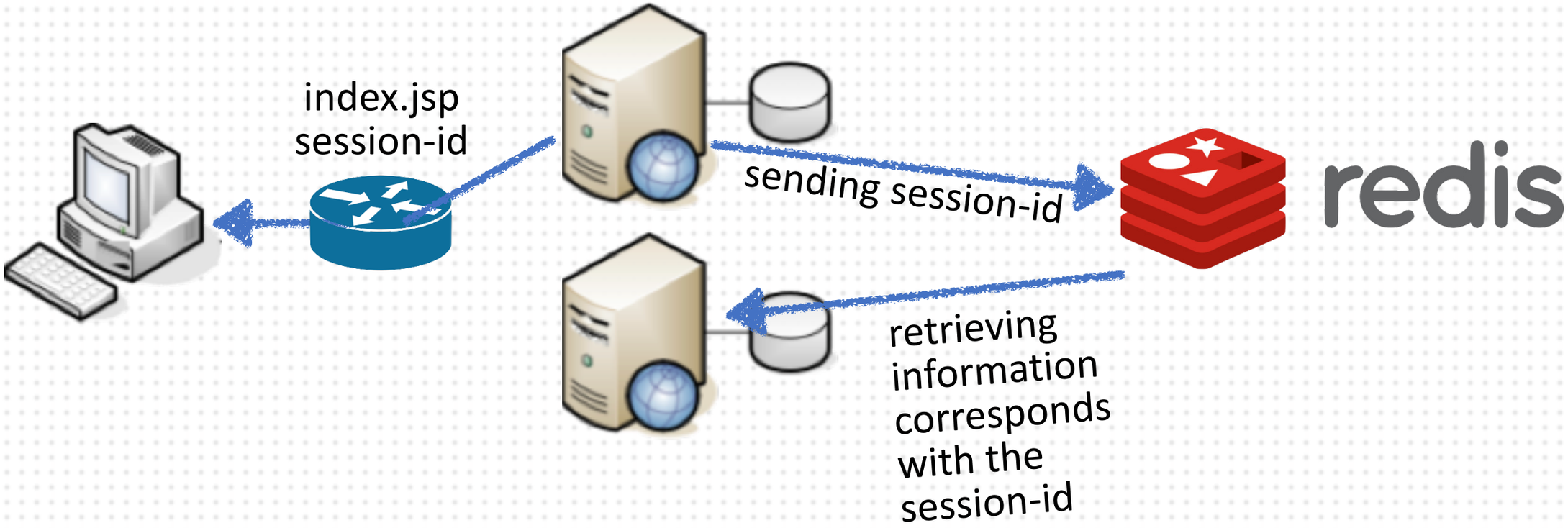
There are some issues

- Session Migration and Server Affinity
  - Determining the session a request belongs to
  - The problem with proxy servers
- System Crash (Client, Server, Network)



Alternatives for Storing Session States

- Using external session storages
  - Can be assumed as standalone hash tables
  - Redis: a NoSQL key-value based storage



Alternatives for Storing Session States

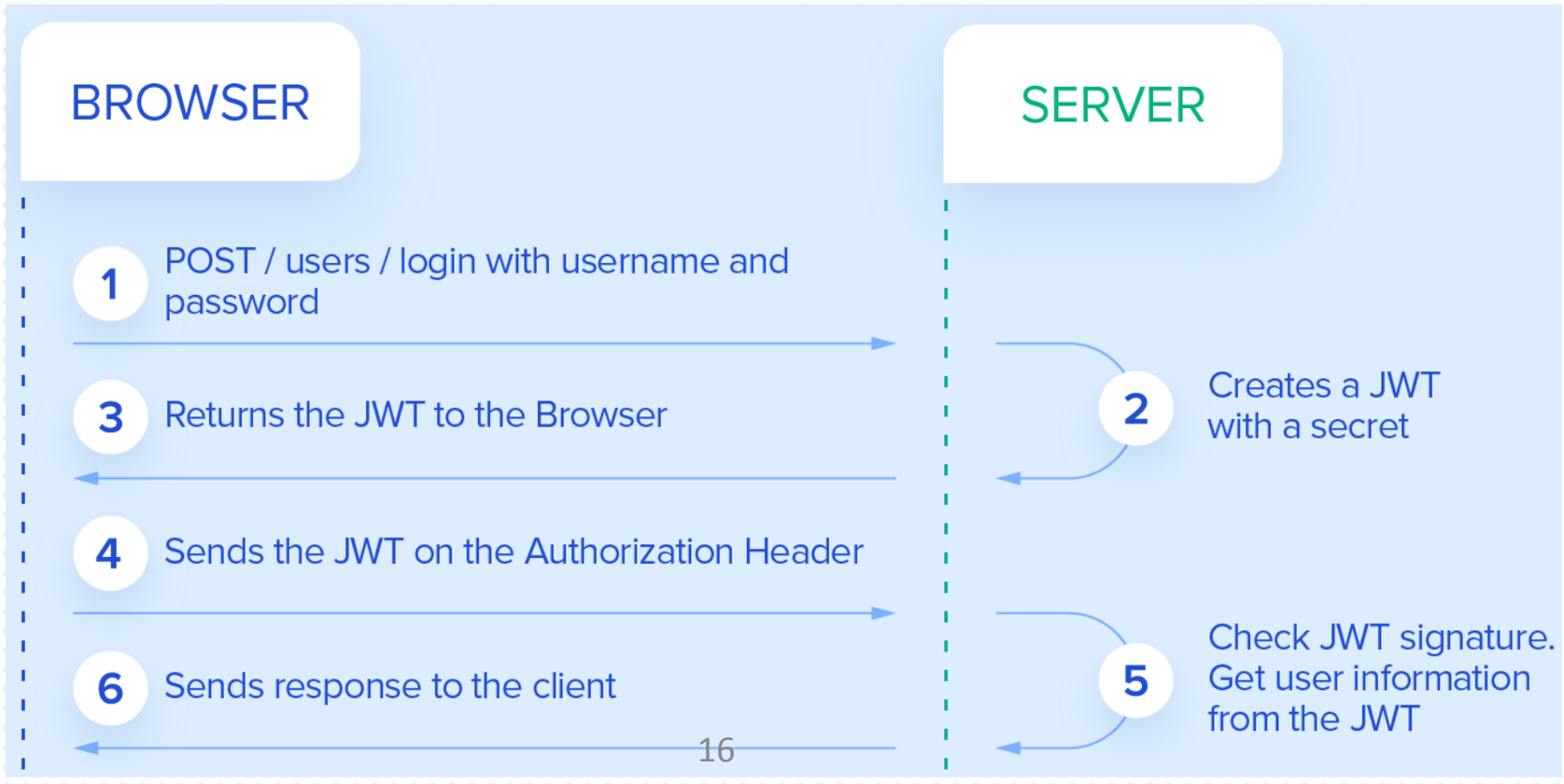
- Using external session storages
  - Can be assumed as standalone hash tables
  - Redis: a NoSQL key-value based storage
- Using Database
  - Separating session storage from the recorded data
  - A separate set of tables
  - Integrity rules
  - Using serialized LOB

Some New Issues!

- Bandwidth
- Latency
- User cancels a request or it just disappears!
  - Killing aged sessions
    - Possible contention on session table
    - Partitioning session tables in 12 segments: every two hours, clear the oldest segment and redirect every insert to it
- Implementation effort
- Mixture of the methods

Sessionless Services!

- The idea of JSON Web Token (JWT)
  - A small digitally signed token
  - All needed information are stored in the token
  - JWT needs to be attached to every HTTP request



JWT Content

- Header: typically includes the type of the token and hashing algorithm.

```
{
  "typ": "JWT",
  "alg": "HS512"
}
```

- Payload: typically includes data about a user and for whom is token issued.
- Signature – it's used to verify if a message wasn't changed along the way.

```
{
  "iss": "sec-api",
  "aud": "sec-app",
  "sub": "user",
  "exp": 1548242589,
  "rol": [
    "ROLE_USER"
  ]
}
```

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzUxMiJ9.eyJpc3MiOiJzZWN1cmUtYXBpIiwiYXVkIjoic2VjdXJl
LWFwcCIsInN1YiI6InVzZXIiLCJleHAiOjE1NDgyNDI1ODksInJvbCI6WyJST0xFX1VTRVIiXX0.GzUPU
WStRofrWI9Ctfv2h-XofGZwcOog9swtuqg1vSkA8kDWLcY3InVgmct7rq4ZU3lxI6CGupNgSazypHoFOA
```

JWT Cons

- No need to store session information
  - Easier load balancing / crash recovery / …
- No need for session garbage collection
- Integrated with a wide range of Java Enterprise frameworks and libraries (including Spring)

JWT Pros

- Commonly used data has to be retrieved from data sources by each request
  - There is no need for them in many cases
- Encryption/Decryption overhead
- What shall the system do in the case of logout operation?